From a Lean source to a reviewable explanation

An architectural companion to Lean Exposition

24 October 2026 · revised draft, not independently reviewed

Draft scope. This companion explains the repository's present file and command contracts. It has not received independent mathematical or reader review. The repository's QAOA/SK papers are a worked example, not a claim that the system has verified their mathematical interpretation.

1. Why keep more than a proof file?

A Lean theorem answers a precise question: whether a term has the type stated in a particular formal environment. A mathematician reading a paper asks additional questions. Are the hypotheses the ones the prose describes? Which source proves the analytic step that Lean does not contain? Does the short account omit a condition retained in the long one? The answers can change while the Lean proof still compiles.

Lean Exposition keeps these questions connected without making them identical. Its local evidence graph records selected declarations, assumptions, cited sources, observations and the relations among them. Three authored texts then serve different readers: a long reasoning record, a shorter mathematical paper, and an expository account. A mapping alongside each text binds passages to the exact graph edition and records the state of review. The repository packages these materials for inspection; it does not generate a proof from prose or certify that the prose means what the author intended.

The SK example makes the distinction concrete. OpenAI's full-support manuscript argues that an admissible minimiser of the zero-temperature Parisi functional for the pure SK model at zero external field has full relative Stieltjes support on $[0, 1)$. A separate scalar-value argument and a further QAOA construction use related mathematics but do not follow merely from the words "full support." The graph can record those distinct claims and their sources. The papers must still explain the analytic bridges and say which steps are formalised, source-checked, conditional or open. The current drafts retain unresolved review obligations; no bundle status turns them into accepted theorems of exposition.

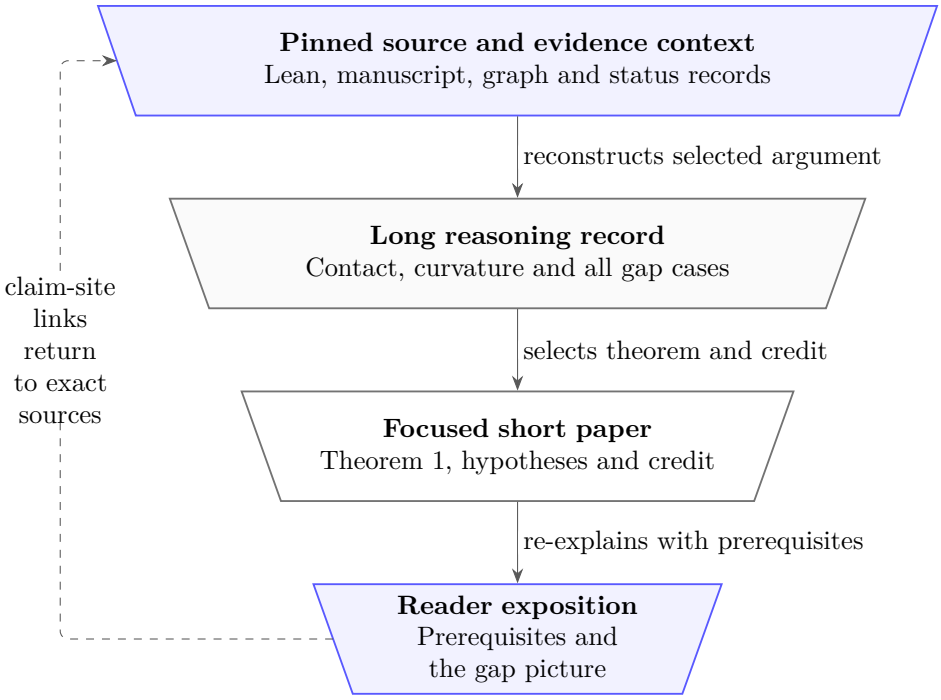


Figure 1: An inverted reader funnel adapted from Will Cook's existing mathematics-map visual. It selects a way into one scoped result; the widths represent editorial selection, not page counts, proof strength or the length of the expository paper. Solid arrows name authoring transitions. The separate dashed route returns a reader from a claim to pinned evidence, without asserting that any text or check certifies mathematical meaning.

The sources have different authors and jobs. OpenAI supplies the two original mathematical manuscripts and the selected Lean development. Earlier papers supply mathematical results and methods credited where used. Prove2Me supplies a design for collaborative formalisation and a separate public theorem service. Our contribution here is the reconstructed accounts, their source correspondence, and the local tools for inspecting and revising them. Neither an explanation nor a platform link transfers authorship of the mathematics.

2. One route from source to reader

The starting material is a chosen, pinned source: for example, a Lean checkout with its toolchain and a mathematical manuscript with a known edition. An author or extractor records relevant declarations and evidence in a graph. A theorem that uses a lemma points to that lemma in a dependency relation. A claim's link to a selected formal statement is a different, formalisation relation; a citation identifies an external source edition. The reader can follow the recorded relation from a claim to its stated support, or ask which other claims refer to a changed item. The meaning and direction of each link remain visible.

An author can work through the SK example in a deliberate order. The long account reconstructs the full support-gap calculation and its dependence on the variation formula. The short paper distils the mathematical argument with its hypotheses visible. The expository account explains why a hypothetical gap conflicts with the moment identity before returning to the formal statement. These are authored stages, not automatically generated summaries. Their emphases differ, but a

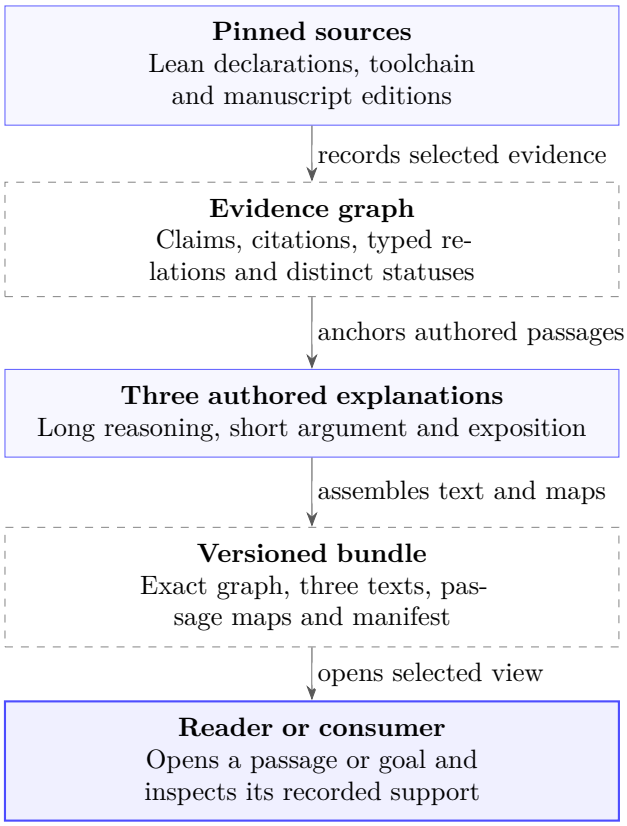


Figure 2: Production and inspection workflow. The bundle includes the exact graph edition as well as authored papers and maps; its byte checks do not judge mathematical meaning. Adapted from the authored-records and derived-views composition in the Plectis Erdős systems paper.

passage map binds each account to the same selected graph edition. If an author changes the source graph or a passage, the corresponding byte hash changes and assembly requires a revised mapping. A successful hash check establishes correspondence to recorded bytes, not that the explanation is true.

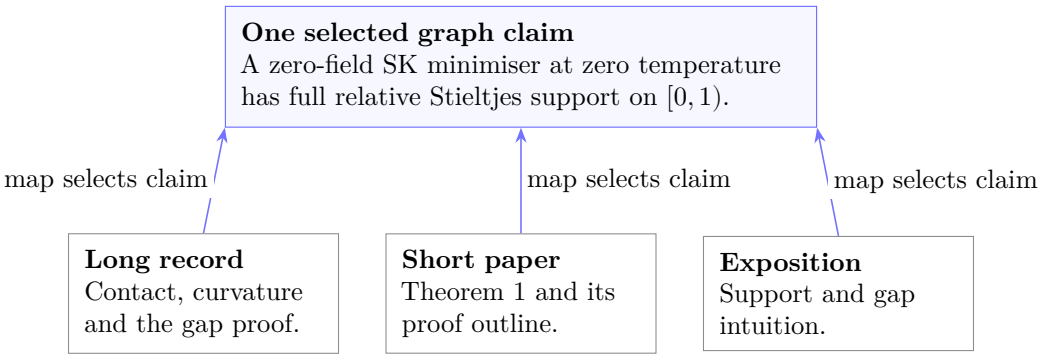


Figure 3: Passage-to-claim correspondence, not a proof dependency. Three authored passages can point to one recorded mathematical claim. The forked composition adapts the shared-support, three-version figure in the Plectis Erdős systems paper.

The graph supports several *typed views* of the same records. A declaration view follows formal Lean dependencies; an argument view shows curated support and formalisation relations with their own meanings and directions; a citation view follows sources; a platform view follows public decomposition metadata; and a provenance view follows recorded source and receipt links. A selected goal determines where traversal starts, without removing unrelated evidence from the underlying graph. A view of no recorded links reports an absence in this graph, not mathematical independence.

A reader may instead start from one paper, a section or a named passage. For example, the short paper's gap argument can be opened beside the contact and curvature steps it uses. The paper view gives the exact text and its recorded correspondence to graph claims, then expands their recorded prerequisites, including assumptions introduced elsewhere. Selecting a smaller piece of prose does not make its conditions disappear. The source-grounded intermediate records remain distinct from the selected formal interface and retain their own evidence status.

Two questions about another account should be kept separate. "Where is this claim mapped?" asks for passages whose maps name it directly. "Which passages use it?" asks for passages whose mapped claims reach it through recorded support relations. The second answer includes a path explaining the match. A passage may rely on an ingredient without explaining it, and a teaching example may explain an idea without being a premise. The query therefore reports recorded correspondence and reachability, not a judgment about what every sentence teaches or proves.

Section selection follows the actual document outline. If a mapped passage crosses a section boundary, the view identifies the overlap and retains the full passage with its support. It also reports text without a passage map, passages outside the selection, required assumptions and unresolved records. Reading order is a document relation; it does not enter the mathematical support traversal. The graph remains partial, so a complete expansion of recorded links is not a complete mathematical

dependency audit.

The three accounts share references to the same graph edition. A front end can switch from the short paper to the long record or exposition while keeping the selected claim in view, and can regenerate its answers from the existing bundle. No second graph or extra bundle member is needed. The architectural companion itself remains a guide outside the bundle's three mathematical paper stages.

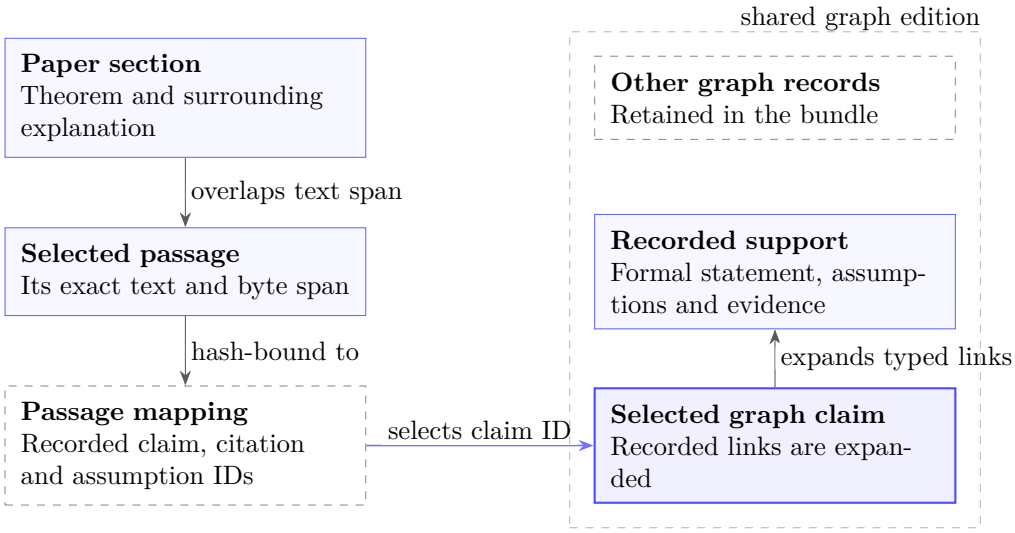


Figure 4: A local paper view follows document structure, passage-to-claim mapping and recorded argument relations without discarding other graph records. The selected support includes typed relations whose meanings and directions remain explicit; this is not a uniform proof graph. The selection-and-support shape adapts the registered-support figure in the Plectis Erdős systems paper.

Following one step through the three accounts

Take the positive-coefficient case of the gap argument. The short paper uses contact at the endpoints and strict convexity of the derivative of a diffusion moment to obtain a contradiction. The long record reconstructs the analytic estimate behind that convexity. The exposition first illustrates the missing area, then defines the moment and derives the argument. These are three useful ways to approach the same step.

A passage query opens the paragraph with its exact hypotheses. Expanding its recorded prerequisites leads to the source-located contact and curvature claims. The latter leads further to the integrated coercive estimate, its rational identity, the sign inequalities and the approximation step. The selected Lean interface remains separately identifiable. Its presence does not certify every intermediate analytic reconstruction or the density corollary added to the short paper.

The difference between a direct match and a prerequisite match is practical. A passage that states the final support theorem may use curvature without teaching the curvature proof. A direct query for the curvature claim finds its mapped explanations. A prerequisite query also finds passages whose recorded argument uses it, and returns the relation path. This lets a future reader interface offer "explain this step" and "show where it is used" as different actions. Both remain limited by the coverage of the authored graph.

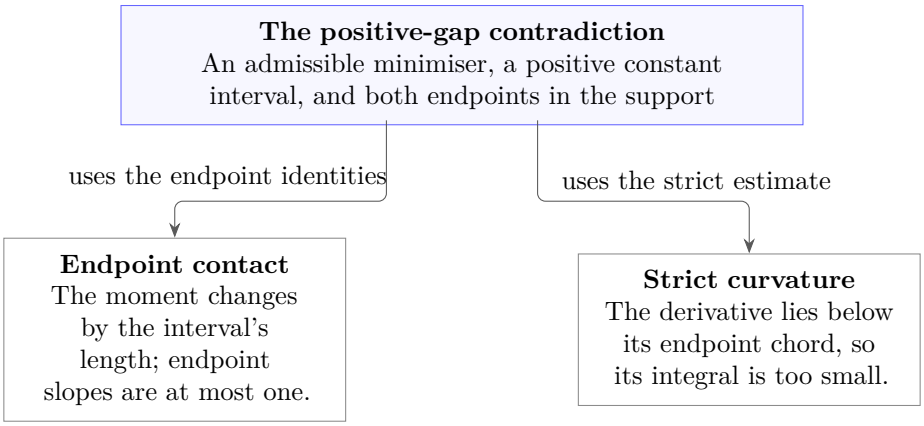


Figure 5: A selected argument view. Arrows point from the contradiction to the ingredients it uses. This is the positive-coefficient bounded-gap branch only; zero coefficients and initial and terminal gaps have separate arguments. The figure adapts the shared-support composition above. Its relations are curated from the original proof, not extracted Lean dependencies.

Cycles may occur among recorded relations. The consumer reports their strongly connected components, the kinds of links within and between them, and a directed acyclic graph of the components. This is a way to navigate mutually connected records; it is not a proof that a circular mathematical argument is valid. Choosing the reverse "consumers" direction can show what refers to a declaration. Neither direction silently changes the original evidence or its status.

The design follows established work on formal mathematics. LeanArchitect's selected collector and output code informed the separation of declaration identity, extracted references and presentation choices; leanblueprint connects mathematical prose with named formal declarations and authored dependencies. The doc-gen4 index and Verso Blueprint provide further examples of source and consumer boundaries. These are documented in the source study, with distinct source revisions. The Lean extractor adapts structural traversal from Comparator, retaining Lean FRO, LLC and Henrik Böving's Apache-2.0 notice in its source. The repository does not claim to have invented these mechanisms.

3. What each check can say

The formal developments depend on Lean and mathlib. Their scholarly sources are de Moura and Ullrich's *The Lean 4 Theorem Prover and Programming Language* (2021) and The mathlib Community's *The Lean Mathematical Library* (2020). These credits explain the software foundations; the exact toolchain and dependency pins identify the environment of a particular proof.

Lean, Comparator and Prove2Me answer different questions. A successful Lean build concerns a particular formal statement in a pinned environment. The Comparator runs a separately stated challenge against a selected implementation and axiom policy. Lean's versioned proof-validation documentation explains these checks and their trust assumptions. The research source uses Lean 4.34.1 with a historical standalone Comparator; the later integrated interface does not establish a result for that earlier environment. A platform observation records public metadata at a time: visibility, status, environment and decomposition links. These observations must stay separate. In particular, a platform's "proved" label does not mean this repository replayed the proof, and a Comparator startup failure is not a mathematical counterexample. The QAOA/SK example preserves

distinct observations. An early invocation could not start the Comparator. Later invocations built and exported the full-support and scalar-value challenges, then stopped during their solution builds because imported compiled modules were missing. Neither exported or accepted its solution. This is an operational failure before comparison, not a refutation. A corrected local build configuration is a new input edition and requires its own replay; repairing the configuration does not change the historical outcome.

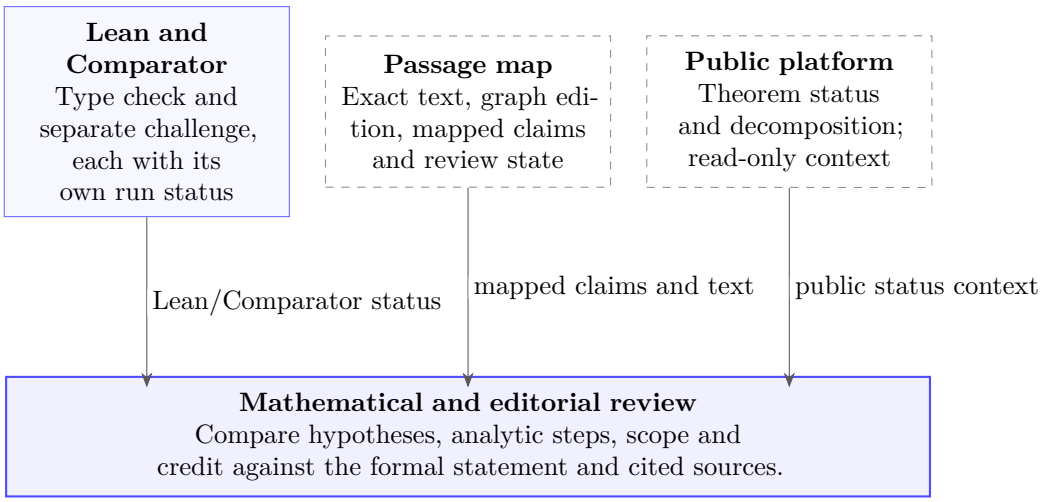


Figure 6: Three distinct records can inform a later human review. Their arrows specify what each channel supplies; none certifies mathematical interpretation or says review has occurred. Adapted from the two-correspondence figure in the Plectis Erdős systems paper.

Extraction also has a narrower meaning than proof acceptance. Direct references in a declaration's type, available proof body and structural metadata are recorded separately. A hidden proof body limits body traversal even when Lean can report axiom metadata from the compiled environment. The request, driver, source set and output belong to the extraction evidence; a changed source set requires a fresh check. A process exit code alone does not identify which named challenges were accepted. The verification guide explains these boundaries and the retained observations.

The system deliberately reports `not_run`, `unverified`, `not_reviewed` and `needs_revision` where they apply. A build, a source hash, a graph closure and a rendered PDF each establish a narrower fact than human comprehension. One can inspect the records now while still asking a mathematician to check the argument.

4. Why the bundle has eight files

The portable bundle has a manifest, the graph, and one Markdown text with a passage map for each of the three stages. The manifest records the format, local publication state and hashes of the other seven files. The graph keeps typed evidence and status. Each map binds its authored text to exact graph bytes, selected goals, claims, citations, assumptions, passages and review state. The pairings keep mathematical prose readable while making its recorded support inspectable.

Removing a map would hide one account's graph correspondence; merging the texts would erase their distinct audiences. Lean source, manuscript editions and verification receipts remain external and are identified by records and digests where available. A consumer validates the directory or

ZIP and reads selected evidence or paper views. The bundle is not a self-contained proof archive, and its `semantic_certification` field is false.

The work's name, its format and its exact edition serve different purposes. The name can remain the same when the work changes. The manifest's SHA-256 identifies one captured edition, including the graph, three texts and their maps. A paper passage has a reference to its text and mapping, while a graph claim has a reference to its graph snapshot. A section's byte offset identifies its location in that particular text. These are precise handles for an edition, not promises that a passage or claim keeps the same meaning after revision.

Suppose an author leaves a paragraph unchanged but changes its mapping from an unverified source argument to a selected formal statement. Its words are the same, but the correspondence now makes a different assertion. The new map and manifest identify a successor, and the earlier review remains attached to the old one. Similarly, a changed hypothesis or source edition must not be hidden behind an unchanged claim name. A query can help find recorded consumers of the changed item; because the graph is partial, that list does not certify that every affected passage has been found.

The review request, original response and author decision belong beside the bundle. If feedback on edition A produces edition B, a later adoption record names both editions and explains what changed. An author recording a correction is different from a reader confirming it. The old response is preserved, and acceptance is not transferred by refreshing hashes. A PDF or HTML representation also needs its own identity: Markdown bytes do not identify a rendered page or its appearance.

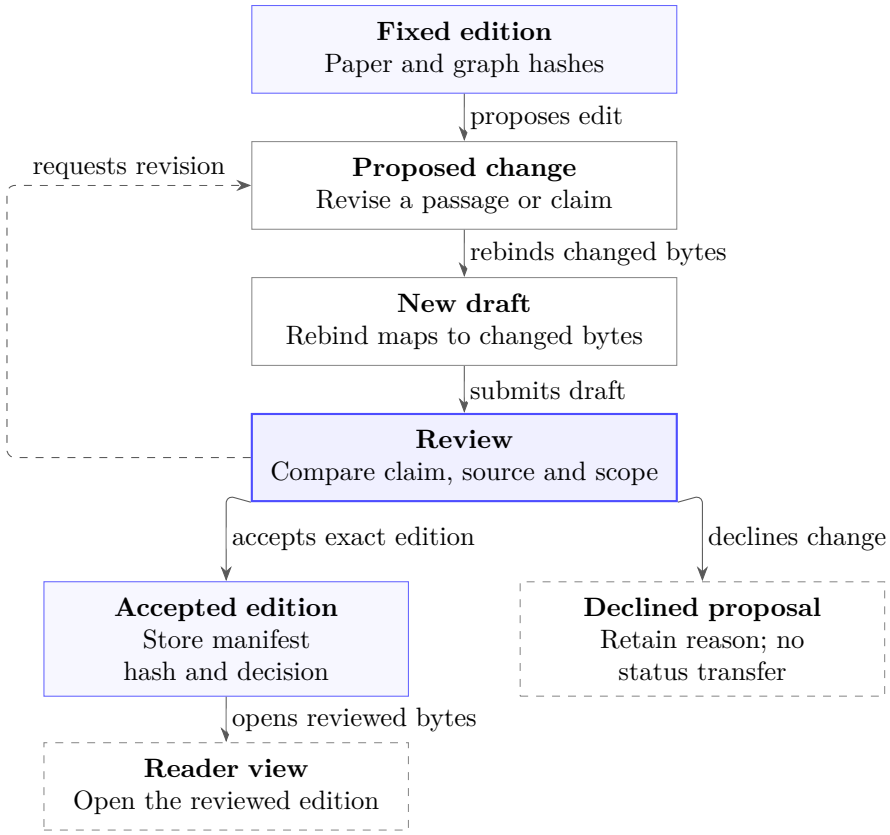


Figure 7: Review-state transitions for an immutable edition. A revision creates new bytes and does not inherit an earlier decision. The accept/revise/decline composition adapts the refinement figure in the Plectis Erdős systems paper. The present QAOA/SK drafts have not been approved.

The reading experience can be simpler. A PDF can lead with the argument and offer source and status links at the relevant claim. A consumer can show a small paper or theorem view, expand dependencies on request, and leave hashes in the manifest. A duplicate derived view can be regenerated from the graph. These changes must preserve source versus interpretation, formal check versus review, and one edition versus another.

5. Use and external handoff

The local command-line tool validates graphs, selects goals, creates empty scaffolds, assembles authored paper-and-map pairs, and checks or inspects a bundle. Paper queries also find direct correspondences or recorded prerequisite uses across the three accounts. Their answers retain the exact edition, matching passages and support boundary. The consumer guide gives commands for a local directory and its exported ZIP. Assembly requires valid existing bindings; it neither writes mathematics nor repairs a stale interpretation.

The optional Prove2Me adapter reads partial public theorem and decomposition metadata using an existing credential and pinned documentation version. It filters private or unknown children; it does not submit proofs, upload papers or receive bundle reviews. The Prove2Me paper separates immutable statements, proof submissions and human audit (Chen, Marwaha, Lu, Yuen and Peng, 2026, §§3–4). This is an analogy, not evidence these papers were attached to a theorem card. An

external paper handoff would need an explicit edition, attachment and feedback contract.

This companion remains a draft without independent human acceptance. The next review should test whether a mathematician can start at a paper claim, find the exact formal and analytic sources, and understand which verification steps are complete without learning the internal file layout first.

Sources. This draft follows the local CLI, graph, bundle and consumer contracts and QAOA/SK example, revised on 24 October 2026. Figure 1 adapts Will Cook's existing mathematics-map video funnel. The remaining figures adapt Will Cook's *A Repository-Based System for Research and Publication* (CC BY 4.0); the adapted compositions are labelled lifecycle, hypothesis, theorem, checks and refinement in that source. The concrete argument view further adapts its shared-support composition. The editorial distinctions also follow the author's public mathematical writing methods. The Prove2Me boundary follows Chen, Marwaha, Lu, Yuen and Peng, *Prove2Me*, §§3–5; its case studies do not evaluate Lean Exposition.